

Le basi

Sintassi di base

Le regole essenziali per scrivere istruzioni PHP leggibili e corrette.

Il tag di apertura

Il codice PHP inizia con:

```
<?php
```

Questo tag avvisa PHP che le righe successive contengono istruzioni da eseguire.

```
<?php  
echo "Ciao";
```

Nei file che contengono solo PHP, di solito non si mette il tag di chiusura `?>`. Evitarlo riduce piccoli errori causati da spazi o righe vuote alla fine del file.

Il punto e virgola

In PHP molte istruzioni finiscono con `;`.

```
<?php  
$nome = "Marta";  
echo $nome;
```

Il punto e virgola dice: questa istruzione è finita. Se lo dimentichi, PHP spesso mostra un errore di sintassi.

Blocchi di codice

Le parentesi graffe `{ }` raccolgono più istruzioni dentro un blocco.

```
<?php
$eta = 20;

if ($eta >= 18) {
    echo "Puoi entrare";
}
```

Qui il messaggio viene mostrato solo se la condizione è vera.

PHP dentro HTML

PHP può vivere dentro una pagina HTML:

```
<h1>Profilo</h1>

<?php
$nome = "Luca";
echo "<p>Ciao, $nome</p>";
?>
```

HTML descrive la struttura della pagina. PHP prepara parti dinamiche.

Spazi e leggibilità

PHP non richiede spazi perfetti, ma il codice leggibile aiuta molto.

```
<?php
$prezzo = 10;
$sconto = 2;
$totale = $prezzo - $sconto;

echo $totale;
```

Una riga vuota tra gruppi di istruzioni rende più chiaro cosa sta succedendo.

Commenti, istruzioni e blocchi insieme

Un piccolo file PHP può contenere tutti questi elementi:

```
<?php
// Dati di partenza
$nome = "Sara";
$eta = 21;

if ($eta >= 18) {
    echo "Ciao $nome, puoi continuare.";
} else {
    echo "Ciao $nome, questa area non è disponibile.";
}
```

La prima parte prepara i dati. La seconda parte controlla una condizione. Il blocco tra `{` e `}` contiene le istruzioni da eseguire se la condizione è vera.

Errore comune: confondere HTML e PHP

HTML non usa il punto e virgola e non esegue calcoli. PHP invece esegue istruzioni.

```
<p>Questo è HTML</p>

<?php
echo "Questo testo viene prodotto da PHP";
?>
```

Quando una pagina contiene entrambi, chiediti sempre: questa riga descrive la pagina o deve essere eseguita come istruzione?

Riepilogo

Per scrivere PHP corretto all'inizio ricorda quattro cose: apri il codice con `<?php`, chiudi le istruzioni con `;`, usa le graffe per i blocchi e tieni il codice leggibile con spazi e nomi chiari.

Commenti

Come usare i commenti per spiegare il codice PHP senza eseguirlo.

Cosa sono i commenti

Un commento è testo scritto per chi legge il codice. PHP lo ignora e non lo esegue.

I commenti servono a spiegare una scelta, ricordare un dettaglio o rendere più chiaro un passaggio non immediato.

Commenti su una riga

Usa `//` per scrivere un commento breve.

```
<?php
// Salviamo il nome dell'utente
$nome = "Giulia";

echo $nome;
```

PHP esegue `$nome = "Giulia";` ed `echo $nome;`, ma ignora la riga che inizia con `//`.

Commenti su più righe

Usa `/*` e `*/` quando il commento occupa più righe.

```
<?php
/*
Questo esempio calcola il prezzo finale
partendo da prezzo iniziale e sconto.
*/
$prezzo = 30;
$sconto = 5;

echo $prezzo - $sconto;
```

Quando usare i commenti

Un buon commento spiega il perché, non ripete l'ovvio.

```
<?php
// Applichiamo lo sconto solo agli ordini sopra i 50 euro
if ($totale > 50) {
    $totale = $totale - 10;
}
```

Questo commento è utile perché spiega la regola.

Quando evitarli

Questo commento non aiuta:

```
<?php
// Stampa il nome
echo $nome;
```

Il codice è già chiaro. Se devi commentare ogni riga, forse puoi scegliere nomi migliori o dividere il codice in pezzi più piccoli.

Commentare una scelta

Un commento diventa prezioso quando spiega una regola che non si vede subito dal codice.

```
<?php
$totale = 62;

// La spedizione e gratuita dagli ordini sopra i 50 euro
if ($totale > 50) {
    $costoSpedizione = 0;
} else {
    $costoSpedizione = 4.90;
}
```

Qui il commento non ripete `if ($totale > 50)` . Spiega la regola del negozio.

Usare i commenti mentre studi

Quando impari, puoi usare i commenti per dividere un esercizio in passaggi.

```
<?php
// 1. Salvo i dati iniziali
$prezzo = 20;
$sconto = 5;

// 2. Calcolo il totale
$totale = $prezzo - $sconto;

// 3. Mostro il risultato
echo $totale;
```

Questo aiuta a leggere il programma come una piccola storia. Con il tempo userai meno commenti, perché i nomi delle variabili e delle funzioni faranno già parte del lavoro.

Prova tu

Prendi un esercizio breve e aggiungi tre commenti: dati iniziali, calcolo, risultato. Poi rileggilo e cancella i commenti che ripetono soltanto quello che il codice dice già.

Tipi di dato

Numeri, stringhe, booleani, null e altri valori fondamentali in PHP.

Cosa significa tipo di dato

Un tipo di dato dice che genere di valore stai usando. Un numero, un testo e un vero/falso non si comportano nello stesso modo.

PHP capisce il tipo dal valore che assegna.

Numeri interi e decimali

Gli interi sono numeri senza virgola.

```
<?php  
$eta = 30;
```

I decimali si scrivono con il punto.

```
<?php  
$prezzo = 12.50;
```

Stringhe

Una stringa è testo. Si scrive tra virgolette.

```
<?php  
$nome = "Luca";  
$messaggio = 'Ciao';
```

Le virgolette doppie permettono di inserire variabili dentro il testo.

```
<?php
$nome = "Luca";
echo "Ciao $nome";
```

Booleani

Un booleano può essere solo `true` o `false`.

```
<?php
$maggiorenne = true;
$pagato = false;
```

Li userai spesso nelle condizioni.

Null

`null` significa: qui non c'è nessun valore.

```
<?php
$telefono = null;
```

Non vuol dire zero e non vuol dire stringa vuota. Vuol dire proprio assenza di valore.

Array

Un array raccoglie più valori in una sola variabile.

```
<?php
$colori = ["rosso", "verde", "blu"];
```


Controllare il tipo

Per osservare un valore puoi usare `var_dump` .

```
<?php
$eta = 30;
var_dump($eta);
```

Output possibile:

```
int(30)
```

`int` indica un numero intero.

Lo stesso valore puo avere tipi diversi

Guarda questi due valori:

```
<?php
$numero = 25;
$testo = "25";

var_dump($numero);
var_dump($testo);
```

Il primo e un numero intero. Il secondo e testo, anche se contiene cifre. Questa differenza conta quando fai confronti, calcoli o validazioni.

Conversioni semplici

A volte un valore arriva come stringa, per esempio da un form. Se vuoi trattarlo come numero, puoi convertirlo.

```
<?php
$etaDalForm = "32";
$eta = (int) $etaDalForm;

var_dump($eta);
```

`(int)` chiede a PHP di trasformare il valore in un intero.

Errore comune: confondere null e stringa vuota

```
<?php
$telefono = null;
$indirizzo = "";
```

`$telefono` non ha valore. `$indirizzo` ha un valore, ma è un testo vuoto. Nei controlli questa differenza può essere importante.

Riepilogo

Prima di usare un valore chiediti che cosa rappresenta: numero, testo, vero/falso, assenza di valore o raccolta di dati. Capire il tipo rende il codice più prevedibile.

Il primo programma PHP

Come creare ed eseguire un primo file PHP partendo da zero.

Creare il file

Apri la cartella degli esercizi e crea un file chiamato `ciao.php`.

Scrivi questo codice:

```
<?php  
echo "Ciao, PHP!";
```

La riga `<?php` dice: da qui inizia codice PHP. La parola `echo` mostra un testo.

Eeguire il file dal terminale

Apri il terminale nella stessa cartella del file e scrivi:

```
php ciao.php
```

Output:

```
Ciao, PHP!
```

PHP legge il file ed esegue le istruzioni dall'alto verso il basso.

Aggiungere una seconda istruzione

Modifica il file così:

```
<?php  
echo "Ciao, PHP!\n";  
echo "Questo e il mio primo programma.";
```

`\n` crea una nuova riga nel terminale.

Output:

```
Ciao, PHP!  
Questo e il mio primo programma.
```

Usare il server locale integrato

PHP può anche avviare un piccolo server locale. Serve quando vuoi vedere una pagina nel browser.

Crea `index.php` :

```
<?php  
echo "<h1>Ciao dal browser</h1>";
```

Poi esegui:

```
php -S localhost:8000
```

Apri `http://localhost:8000` nel browser. Vedrai il titolo prodotto da PHP.

Nota: il server locale serve per studiare e provare. Per pubblicare un sito online servono impostazioni diverse.

Capire cosa hai appena fatto

Hai usato PHP in due modi diversi.

Nel primo caso hai eseguito un file dal terminale. Questo è utile per esercizi piccoli, prove veloci e script che non devono mostrare una pagina web.

Nel secondo caso hai avviato un server locale. Il browser ha chiesto una pagina a `localhost:8000`, PHP ha eseguito `index.php` e ha mandato al browser il risultato.

`localhost` significa "questo computer". Non è un sito pubblico: è un indirizzo speciale che usi per provare il progetto sulla tua macchina.

Errore comune: aprire il file direttamente

Se apri `index.php` facendo doppio clic sul file, il browser potrebbe mostrarti il codice o non eseguirlo correttamente. Per eseguire PHP serve passare da PHP stesso, con il comando:

```
php -S localhost:8000
```

Poi apri l'indirizzo nel browser.

Prova tu

Modifica il messaggio dentro `echo` , salva il file e ricarica la pagina. Se il testo cambia, hai completato il ciclo base: scrivi, salvi, esegui, osservi il risultato.

Installazione e ambiente

Come installare PHP, scegliere un editor e verificare che l'ambiente funzioni.

Cosa ti serve

Per scrivere PHP ti servono tre cose:

1. PHP installato sul computer
2. un editor di testo per scrivere codice
3. un terminale per eseguire i comandi

L'editor può essere Visual Studio Code, PhpStorm o un altro programma simile. Il terminale è la finestra dove scrivi comandi come `php --version` .

Installare PHP

Su macOS puoi usare Homebrew:

```
brew install php
```

Su Windows puoi installare PHP da [php.net](https://www.php.net) oppure usare un pacchetto locale come XAMPP. Su Linux di solito puoi usare il gestore pacchetti della distribuzione.

Non importa il metodo scelto: alla fine il comando `php` deve funzionare nel terminale.

Verificare l'installazione

Apri il terminale e scrivi:

```
php --version
```

Dovresti vedere una risposta simile:

```
PHP 8.3.0
```

Il numero può essere diverso. L'importante è che il terminale riconosca PHP e mostri una versione.

Scegliere una cartella di lavoro

Crea una cartella per gli esercizi, per esempio `corso-php`. Dentro questa cartella metterai i file `.php`.

```
mkdir corso-php  
cd corso-php
```

`mkdir` crea la cartella. `cd` entra nella cartella.

Editor consigliato

Un buon editor ti aiuta con colori, indentazione e segnalazione degli errori. Visual Studio Code va benissimo per iniziare.

Suggerimento: salva sempre i file PHP con estensione `.php`, per esempio `index.php`. Senza questa estensione, PHP non sa che deve eseguire il file come codice.

Se il comando non funziona

Se il terminale risponde con un messaggio simile a `command not found` o `php non e riconosciuto`, PHP non e ancora raggiungibile dal terminale.

Questo non significa che hai sbagliato tutto. Di solito vuol dire una di queste cose:

- PHP non e stato installato
- il terminale va chiuso e riaperto
- il percorso di PHP non e stato aggiunto al `PATH`

Il `PATH` e una lista di cartelle in cui il terminale cerca i programmi. Quando scrivi `php`, il terminale guarda in quelle cartelle e prova a trovare l'eseguibile di PHP.

Piccola verifica finale

Crea un file chiamato `prova.php` con questo contenuto:

```
<?php  
echo "PHP funziona";
```

Poi esegui dalla stessa cartella:

```
php prova.php
```

Se vedi `PHP funziona`, l'ambiente e pronto. Nel prossimo passo potrai concentrarti sul codice, non sull'installazione.

Operatori

Come usare operatori aritmetici, di confronto, logici e di assegnazione.

Cosa sono gli operatori

Gli operatori sono simboli che fanno qualcosa con i valori: sommano, confrontano, assegnano o combinano condizioni.

Operatori aritmetici

Servono per fare calcoli.

```
<?php
$prezzo = 20;
$ sconto = 5;

echo $prezzo - $sconto;
```

Operatori comuni:

- + somma
- - sottrazione
- * moltiplicazione
- / divisione
- % resto della divisione

```
<?php
echo 10 % 3;
```

Output:

```
1
```

Assegnazione

= mette un valore in una variabile.

```
<?php
$totale = 15;
```

Puoi aggiornare un valore in modo breve:


```
<?php
$totalale = 10;
$totalale += 5;

echo $totalale;
```

Confronto

Gli operatori di confronto producono `true` o `false` .

```
<?php
$eta = 18;
var_dump($eta >= 18);
```

Operatori comuni:

- `==` uguale come valore
- `===` uguale come valore e tipo
- `!=` diverso
- `>` maggiore
- `<` minore
- `>=` maggiore o uguale
- `<=` minore o uguale

Suggerimento: preferisci `===` quando vuoi un confronto preciso.

Operatori logici

Servono a unire condizioni.

```
<?php
$eta = 20;
$haBiglietto = true;

if ($eta >= 18 && $haBiglietto) {
    echo "Puoi entrare";
}
```

`&&` significa "e". `||` significa "oppure". `!` nega una condizione.

Le parentesi aiutano a rendere chiaro l'ordine dei controlli.

Precedenza: chi viene calcolato prima

PHP segue un ordine nei calcoli. Moltiplicazione e divisione vengono prima di somma e sottrazione.

```
<?php
echo 2 + 3 * 4;
```

Output:

14

Prima viene calcolato `3 * 4`, poi viene aggiunto `2`.

Se vuoi un ordine diverso, usa le parentesi:

```
<?php
echo (2 + 3) * 4;
```

Output:

20

Errore comune: usare = al posto di ===

= assegna un valore. === confronta due valori.

```
<?php
$eta = 18;

if ($eta === 18) {
    echo "Hai 18 anni";
}
```

Dentro una condizione quasi sempre vuoi confrontare, non assegnare.

Prova tu

Crea `$prezzo`, `$quantita` e `$sconto`. Calcola il totale usando almeno un operatore aritmetico e poi controlla con `if` se il totale e maggiore di 50.

Variabili e costanti

Come salvare valori in PHP con variabili e costanti.

Variabili

Una variabile e una specie di scatola con un'etichetta. Dentro puoi mettere un valore e riprenderlo piu tardi.

In PHP le variabili iniziano con `$`.

```
<?php
$nome = "Luca";
$eta = 25;

echo $nome;
```

`$nome` contiene il testo `"Luca"` . `$eta` contiene il numero `25` .

Cambiare valore

Una variabile può cambiare durante il programma.

```
<?php
$totalale = 10;
$totalale = $totalale + 5;

echo $totalale;
```

Output:

15

PHP prende il vecchio valore di `$totalale` , aggiunge `5` e salva il nuovo valore nella stessa variabile.

Nomi validi

Un nome di variabile può contenere lettere, numeri e `_` , ma non può iniziare con un numero.

```
<?php
$nomeUtente = "Sara";
$prezzo_finale = 19.90;
```

Scegli nomi chiari. `$prezzo` è meglio di `$p` .

Costanti

Una costante salva un valore che non dovrebbe cambiare.

```
<?php
const IVA = 22;

echo IVA;
```

Le costanti non usano `$` . Per convenzione spesso hanno nomi in maiuscolo.

Puoi anche usare `define` :

```
<?php
define("NOME_SITO", "Manuale PHP");

echo NOME_SITO;
```

Usa una variabile quando il valore puo cambiare. Usa una costante quando il valore deve restare fisso.

Variabili nei calcoli

Le variabili diventano utili quando il valore viene riusato piu volte.

```
<?php
$prezzo = 40;
$ sconto = 8;
$totale = $prezzo - $sconto;

echo "Totale: $totale euro";
```

Se domani il prezzo cambia, modifichi solo `$prezzo` . Il calcolo continua a funzionare.

Errore comune: dimenticare il dollaro

In PHP `$nome` e `nome` non sono la stessa cosa.

```
<?php
$nome = "Luca";
echo nome;
```

Questo codice è sbagliato perché `nome` senza `$` non indica la variabile. Quando vuoi leggere o modificare una variabile, usa sempre il dollaro.

Scegliere tra `const` e `define`

Per iniziare, usa `const` quando dichiari costanti semplici nei tuoi file.

```
<?php
const COSTO_SPEDIZIONE = 4.90;
```

`define` esiste ancora e lo incontrerai in molti progetti, ma `const` è più leggibile nei casi base.

Prova tu

Crea tre variabili: `$nome`, `$prezzo` e `$quantita`. Calcola il totale e stampa una frase come `Luca deve pagare 30 euro`.